XM · CLASE 11

Soluciones de Analítica con Databricks

Clase 11: Del modelo registrado a quien lo usa: batch, Serving y Apps

Docente: Manuela Larrea Gómez

Agenda de la clase 11

2 h · cuatro canales para un pronóstico, cómo escoger, y verlo funcionando con el modelo de la clase 9

01	20 MIN	**Teoría, parte 1** ¿Quién consume el pronóstico? Cuatro canales y tres preguntas para escoger · inferencia batch que se puede auditar · MERGE idempotente y challenger en sombra
02	10 MIN	**Receso**
03	20 MIN	**Teoría, parte 2** Model Serving: endpoint, scale to zero, traffic split, AI Gateway · Databricks Apps como principal de Unity Catalog · dashboards y Genie · sector
04	65 MIN	**En acción: notebook guiado + lab** pronostico_demanda con linaje · vista observado vs. estimado · endpoint compartido · la app lee tu vista si le das permiso
05	5 MIN	**Cierre** Hito 5 cerrado · tag s11 · así sería en Azure · próxima clase: Lakeflow Jobs

Un modelo que nadie consume no existe todavía

Clases 4–10 construyeron el modelo y su evidencia. Hoy llega a sus consumidores: se cierra el hito 5

355
series de demanda con contrato en oro (clases 4–8)

7 runs
en el experimento demanda_7d (clase 9)

4,7 %
WAPE promedio walk-forward del @champion (LightGBM global)

0
personas de XM que lo han usado hasta hoy

Lo que hay en el registro

workspace.c01_<usuario>.demanda_7d · v1 @champion lightgbm_global · v2 @challenger naive_festivo · firma de 23 columnas · código empacado · linaje a oro versión N

Lo que necesita un analista de XM

Un número, para un día, para una serie, donde ya trabaja. Y saber qué modelo lo produjo y si sigue acertando.

La pregunta de hoy es de producto antes que técnica: ¿quién consume el pronóstico, cada cuánto y con qué información en la mano?

Un pronóstico, cuatro canales

Se escoge por el consumidor, no por la tecnología. La app y el dashboard no llaman al modelo: leen la tabla del batch

demanda_7d @champion

Modelo registrado en Unity Catalog (clase 9): pyfunc, firma de 23 columnas, código empacado, linaje a oro.

Features: features_demanda_diaria, una vez al día.

1 · Batch

minutos · 1 vez al día

Job que pronostica todas las series y escribe una tabla Delta. Los demás canales la leen.

Planeación de la operación, informes, otros modelos. El canal por defecto.

2 · Serving

milisegundos

Endpoint REST: alguien manda filas y recibe la predicción en el momento. El que llama trae las features.

Un sistema: simulador "¿y si entra una carga de 50 MW?"

3 · App

interactiva

Databricks App (Streamlit, Dash, Gradio): filtros, gráficas, decisiones. Lee la tabla del batch.

Analistas de demanda, área comercial.

4 · AI/BI

interactiva

Dashboard para lo que se mira siempre; Genie para lo que se pregunta una vez, en español.

Gerencia, seguimiento semanal.

El modelo corre una vez; todos consumen el mismo resultado. Más barato, más auditable y consistente: el gerente y el analista ven la misma cifra.

Tres preguntas para escoger el canal, con el caso de XM

Si las features cambian una vez al día, un endpoint no hace el pronóstico más fresco: solo más caro

¿Cada cuánto cambia la información?

Oro se actualiza 1 vez al día → el pronóstico también. Tiempo real no agrega frescura.

¿Quién trae las features?

Batch: el job, desde oro. Endpoint: quien llama. Un escenario "¿y si entran 50 MW?" lo trae el cliente.

¿Persona o sistema?

Persona que explora: app o dashboard. Sistema: tabla o endpoint.

Consumidor en XM	Qué necesita	Canal	Encendido 24 h
Planeación de la operación	Las 355 series × 7 días, cada mañana	Batch → tabla Delta	No
Área comercial	Explorar por agente por qué se desvió la semana	App (lee la tabla)	No (se enciende al usarla)
Gerencia	Un número por semana y su tendencia	Dashboard AI/BI · Genie	No
Simulador de escenarios	Pronóstico con una carga hipotética, en el momento	Endpoint REST	Sí (o scale to zero)

Inferencia batch: seis pasos que un auditor puede seguir

La lógica vive en src/xm_demanda/inference/batch.py; la usan el lab y el job de producción (notebooks/05_batch_inference.py)

1
Resolver el alias
@champion → versión N, una sola vez al inicio
¿Qué modelo produjo la cifra?

2
Leer oro
Con su versión Delta (versionAsOf) y el contrato verificado
¿Con qué datos?

3
Escoger filas
Últimos 7 días (producción) o desde el corte (backtest)
¿Qué se pronosticó?

4
Predecir
El mismo pyfunc registrado: misma firma, mismo código
¿Es el modelo evaluado?

5
Validar
Una predicción por fila, sin nulos, sin negativos; si no, falla
¿Se puede publicar?

6
MERGE
Llave (fecha_pronostico, serie, modelo_version)
¿Y si corre dos veces?

pronostico_demanda: el número y su contexto (backtest, oro v15)

emisión	pronóstico	serie	kWh	modelo	v	datos
19-jul	26-jul	ENDD·BOGOTÁ·Reg	29,39 M	lightgbm	1	v15
19-jul	26-jul	ENDD·BOGOTÁ·Reg	29,02 M	naive_festivo	2	v15

Champion y challenger en la misma tabla, con su versión: comparación en sombra sin desplegar nada.

¿Por qué no un endpoint?

355 series × 7 días = 2.485 pronósticos al día con features que cambian una vez al día. Un job de un minuto los produce todos. Para millones de filas: mlflow.pyfunc.spark_udf, o ai_query() en SQL contra un endpoint.

Idempotencia: el mismo job dos veces, la misma tabla

Reintentos, relanzamientos y backfills van a pasar. La versión del modelo en la llave deja convivir champion y challenger

append

corrida 1 → +17.650 filas

corrida 2 (reintento) → +17.650 filas

35.300

El tablero de mañana muestra el doble de demanda. No falla nada: nadie se entera.

MERGE sobre la llave

corrida 1 → 17.650 insertadas · 0 actualizadas

corrida 2 → 0 insertadas · 17.650 actualizadas

17.650

Mismas filas, generado_en actualizado. Una versión nueva agrega sus propias filas.

```
batch.sql_merge_pronosticos()
MERGE INTO pronostico_demanda AS t
USING pronosticos_nuevos AS s
ON  t.fecha_pronostico =
s.fecha_pronostico
AND t.<llave de la serie> = s.<…>
AND t.modelo_version = s.modelo_version
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Se comprueba, no se supone

DESCRIBE HISTORY guarda numTargetRowsInserted y numTargetRowsUpdated de cada MERGE. Segunda corrida: 0 insertadas.

Challenger en sombra

Pronostica todos los días al lado del champion; nadie lo consume. En semanas hay evidencia real para promoverlo o no.

RECESO

10 minutos

Al volver: Model Serving · Databricks Apps y sus permisos · dashboards y Genie · el sector

Mientras tanto: Git folder en main → Pull · rama feature/c11-serving · anota el ID de la app para el widget app_sp

iData
Global Data & AI

Model Serving: el modelo registrado detrás de una URL

POST /serving-endpoints/xm-demanda-7d/invocations · serverless · gobernado por Unity Catalog

Cliente

Simulador, otra app, un notebook. Necesita CAN QUERY y trae las 23 features.

Un modelo mal empacado no se puede servir: lo que registramos en la clase 9 es lo que hoy construye el contenedor.

→

Endpoint xm-demanda-7d

Served entity

workspace.c01_docente.demanda_7d v1 · Small · scale to zero

Traffic split (opcional)

90 % @champion / 10 % @challenger: prueba A/B sin tocar al cliente

AI Gateway

permisos · límites de uso · tablas de inferencia (cada request y response en Delta)

Contenedor

construido desde el run: MLmodel + requirements (lightgbm) + code_paths (xm_demanda)

Scale to zero

Sin tráfico no cuesta; la primera llamada espera el arranque. Con SLA: réplica mínima.

Free Edition

Solo CPU y pocos endpoints activos por cuenta → uno compartido para el grupo.

Features en tiempo real

Si el cliente solo tiene la llave: feature serving / Online Feature Store (Lakebase) en ms.

Llamar al endpoint: el cliente trae las features

Mismo pyfunc, misma firma: si falta una columna, el endpoint rechaza la solicitud antes de predecir

```
lab 11 · sección 6
from mlflow.deployments import get_deploy_client
cliente = get_deploy_client("databricks")

cuerpo = batch.payload_endpoint(muestra)   # fechas → ISO, NaN → null
r = cliente.predict(endpoint="xm-demanda-7d", inputs=cuerpo)
batch.leer_predicciones(r)

# {"dataframe_split": {"columns": [23 columnas],
#                      "data": [["2026-07-19", "ENDD", …]]}}
# → {"predictions": [{"demanda_estimada_kwh": 29392130.4}]}
```

Latencia y cold start

Tres llamadas seguidas: la primera puede tardar minutos si el endpoint estaba en cero; las siguientes, decenas de milisegundos.

¿Coincide con tu batch?

El endpoint sirve el modelo de la docente; ustedes, el suyo. Mismo código y mismos datos → misma predicción. Si difiere, mira la versión de oro.

Free Edition (hoy)	Plataforma completa
Solo CPU · pocos endpoints activos por cuenta → uno compartido	GPU, provisioned throughput, route optimization, réplicas mínimas por SLA

Databricks Apps: la app es un principal más de Unity Catalog

Contenedor serverless con identidad propia (service principal). Ve exactamente lo que se le concede

Usuario

Entra a la URL con su sesión de Databricks. Sin cluster ni notebook.

→

xm-observado-vs-estimado

app/app.py (Streamlit). Identidad: service principal. Recurso: SQL warehouse sql-warehouse (CAN USE). Free Edition: 3 apps, 24 h.

→

- c01_ana · USE SCHEMA + SELECT sobre la vista → aparece en el selector
- c01_docente · mismos dos GRANT → aparece
- c01_luis · sin GRANT → information_schema no lo devuelve: no existe para la app
- Oro y pronostico_demanda: invisibles. La vista lee con los permisos de su dueño

Autorización de app (hoy)

La app lee con su service principal: todos los usuarios ven lo que ve la app. Simple; útil cuando los datos no distinguen usuarios.

Autorización de usuario (on-behalf-of)

La app actúa en nombre de quien la abre: cada persona ve con sus permisos. En XM, si un agente no debe ver las series de otro.

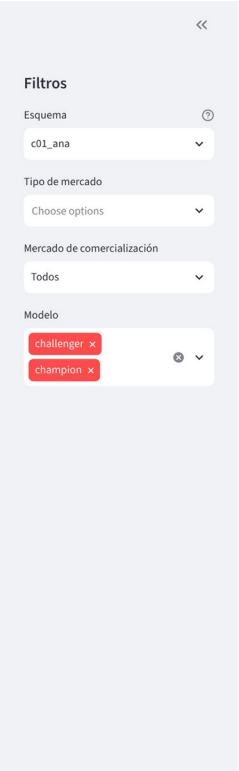
La app del curso: observado vs. estimado por segmento

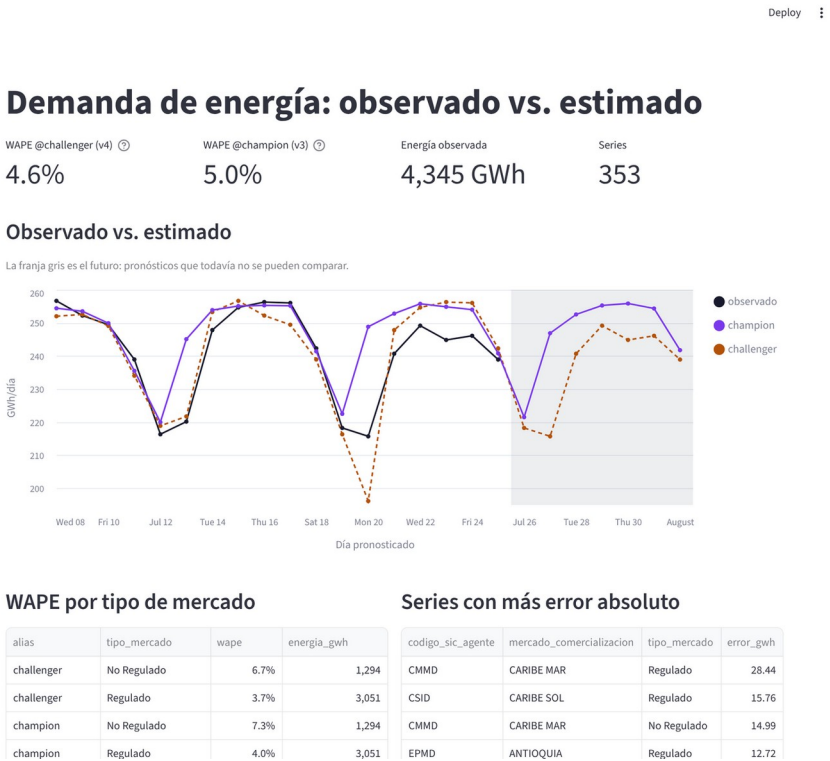
app/ del repositorio · Streamlit · lee v_observado_vs_estimado de cada esquema que le dio permiso

```
# app/app.yaml
command: ["streamlit", "run",
          "app.py"]
env:
  - name: XM_WAREHOUSE_ID
    valueFrom: sql-warehouse
```

20 de julio, festivo

El champion se pasa por arriba; la regla de festivo, por abajo. Ninguno acierta, por razones opuestas. Un WAPE no lo cuenta; la gráfica sí.





alias	tipo_mercado	wape	energia_gwh
challenger	No Regulado	6.7%	1,294
challenger	Regulado	3.7%	3,051
champion	No Regulado	7.3%	1,294
champion	Regulado	4.0%	3,051

codigo_sic_agente	mercado_comercializacion	tipo_mercado	error_gwh
CMMD	CARIBE MAR	Regulado	28.44
CSID	CARIBE SOL	Regulado	15.76
CMMD	CARIBE MAR	No Regulado	14.99
EPMD	ANTIOQUIA	Regulado	12.72

AI/BI: dashboards y Genie sobre la misma vista

Los dos leen la misma vista que la app: una sola definición de WAPE, tres formas de consumirla

Dashboard AI/BI

Datasets en SQL sobre v_observado_vs_estimado; widgets (líneas, contadores, barras), filtros, programación y envío por correo. Permisos de Unity Catalog.

Ejemplo: WAPE de la semana por mercado · observado vs. estimado · top 10 series con más error.

Para: seguimiento semanal, gerencia. Preguntas conocidas de antemano.

Genie

Preguntas en español → SQL sobre las tablas que se le dan. Se le enseña el negocio con instrucciones, consultas de ejemplo y benchmarks.

```
-- ¿En qué mercados el champion tiene más error?
SELECT mercado_comercializacion,
  sum(error_abs_kwh)
  / sum(abs(demanda_observada_kwh)) AS wape
FROM v_observado_vs_estimado
WHERE alias = 'champion'
  AND demanda_observada_kwh IS NOT NULL
GROUP BY ALL ORDER BY wape DESC
```

Genie es tan bueno como lo que se le enseña. Sin instrucciones suele promediar porcentajes por fila, el error de la clase 9. Regla: siempre leer el SQL que generó.

El sector: el pronóstico como servicio interno

El patrón común no es el algoritmo: modelo en batch sobre el lakehouse, resultados consumidos por tablas, tableros y apps con el mismo gobierno

AEMO (Australia)

Operador del mercado eléctrico australiano; entre los primeros adoptantes de la plataforma para energía de Databricks.

Octopus Energy (Reino Unido)

Procesa los datos de medidores inteligentes para entender el consumo de sus clientes.

XM (Colombia)

Publica un pronóstico oficial de demanda. Varios consumidores internos, cada uno con su canal.

Para XM	Canal propuesto	Lo que lo alimenta
Planeación de la operación	Tabla batch diaria	Job de la clase 12, después de gold_features
Seguimiento de demanda	App o dashboard sobre la vista	pronostico_demanda + lo observado
Simulación de escenarios	Endpoint (si existe el caso de uso)	Features que trae el simulador o un feature store en línea

Fuente: Databricks, "Databricks Launches Data Intelligence Platform for Energy", comunicado de prensa, 9 de abril de 2024 · xm.com.co/consumo/pronostico-de-demanda

Lab 11: publicar, servir y dejar que la app te lea

clases/c11-serving-apps/notebooks/lab11_inferencia_serving_apps · widgets usuario y app_sp · rama feature/c11-serving

1

Batch con linaje

Resolver @champion y @challenger, leer oro con versión, predecir ~8.800 filas por modelo

2

TODO 1: preparar_pronosticos

src/xm_demanda/inference/batch.py: columnas, metadatos, sin negativos, falla con nulos

3

Tabla y MERGE dos veces

Segunda corrida: 0 insertadas. Vista observado vs. estimado = WAPE del run de la clase 9

4

Endpoint y app

5 filas al endpoint compartido · TODO 2: dos GRANT al service principal de la app

Criterios de aceptación

- pronostico_demanda con 12 columnas, sin duplicados
- Champion y challenger escritos
- WAPE de la vista = WAPE del run (±0,002)
- Endpoint consultado; tu esquema visible en la app
- Dos preguntas; verificar() en OK; PR en verde

Comparar con s11

github.com/<docente>/Databricks-to-XM/compare/s11...<tú>:Databricks-to-XM:main · o git diff s11 -- src/xm_demanda/inference

Duración: 40 min guiado + 25 min solo

Así sería en Azure: la misma lógica, con garantías de producción

Hoy (Free Edition)	En XM (Azure Databricks)
Un endpoint y una app compartidos por cuenta	Un endpoint por ambiente, desplegado por el bundle (clase 13) con service principal
Scale to zero, solo CPU	Réplicas mínimas según SLA; GPU si el modelo lo pide; route optimization
Sin tablas de inferencia	AI Gateway con tablas de inferencia → monitoreo de la clase 15
App con autorización de app	Autorización de usuario si cada agente solo ve sus series; Private Link
Inferencia corrida a mano	Tarea batch_inference del job diario (clase 12), con reintentos seguros gracias al MERGE

Lo que no cambia: batch por defecto, cada cifra con su linaje, escritura idempotente y permisos mínimos para quien consume.

Resumen de la clase 11 y preparación para la clase 12

Logros de hoy

- **El canal lo decide el consumidor**
  Y casi siempre empieza por batch
- **Cada cifra con su linaje**
  Modelo, versión, datos y hora en cada fila
- **MERGE idempotente**
  Reintentar no duplica; el challenger corre en sombra
- **Serving = mismo modelo, misma firma**
  El cliente trae las features
- **La app ve lo que se le concede**
  Service principal + GRANT mínimos

Próxima clase

- **Lakeflow Jobs**
  ingesta → calidad → features → inferencia en un solo job
- **Reintentos y alertas**
  Seguros porque la escritura es idempotente
- **Parámetros y triggers**
  Por calendario y por llegada de archivo
- **Traer a la clase 12**
  Lab 11 terminado: el job usa su tabla de pronósticos

databricks | Microsoft

iData
Global | Data & AI

Gracias

Nos vemos en la clase 12: Orquestación con Lakeflow Jobs

DOCENTE

Manuela Larrea Gómez